DEVOPS MINI PROJECT · INTERMEDIATE

Netflix-Style Landing Page with Jenkins CI/CD

A complete walkthrough: GitHub → Jenkins → Docker Hub → a live container on AWS EC2

CREATED BY

Mohit Gadilohar

DevOps · CI/CD · Cloud

LinkedIn: www.linkedin.com/in/mohit-gadilohar-devops

Freestyle Job	GitHub	Docker Hub	AWS EC2

This is a learning project with an original, Netflix-inspired design. It is not the actual Netflix application or its proprietary source code.

OVERVIEW

Project at a Glance

In this project you build a Netflix-inspired landing page, keep its source in GitHub, package it as a Docker image using a Jenkins Freestyle job, push that image to Docker Hub, and run it as a container on your own AWS EC2 instance. The guide starts from launching the EC2 server and installing Jenkins, so no prior setup is needed.

How the pipeline flows

1	2	3	4	5	6
Developer HTML, CSS, Git	**GitHub** Source code	**Jenkins Job** Build, test, push, deploy	**Docker Hub** Image registry	**EC2 + Docker** Running container	**Browser** Port 8081

What you will learn

Skill	Where it is covered
Launch and configure an AWS EC2 instance	Phase 1
Install Jenkins and open its web UI	Phase 2
Install Git, Docker and required plugins	Phases 3 and 6
Create a GitHub repository and push the website source	Phase 4
Authenticate securely with Docker Hub	Phases 5 and 6
Create a Jenkins Freestyle project and build script	Phases 7 and 8
Build, test, push and deploy the Docker image	Phases 8 and 9
Trigger the change-and-rebuild loop	Phase 10

Your 10-phase roadmap

#	Phase	Outcome
1	Launch EC2	A running Amazon Linux 2023 server with a locked-down security group
2	Install Jenkins	Jenkins dashboard reachable on port 8080
3	Git + Docker	Git and Docker installed; Jenkins allowed to use Docker
4	GitHub repo	index.html and Dockerfile pushed to GitHub
5	Docker Hub	Image repository and access token ready
6	Credentials + plugins	Git and Credentials Binding plugins; secure Docker Hub login stored
7	Freestyle job	Job created, linked to GitHub, secrets bound
8	Build script	Six-stage script: verify, build, test, login, push, deploy
9	Run + verify	Successful build and live site on port 8081
10	Test CI/CD	Change, push and rebuild to see the loop in action

Quick reference

Item	Value
EC2 instance name	Mohit-Jenkins-CICD
AMI / size	Amazon Linux 2023, t3.medium, 20-30 GiB
Security group	Jenkins-CICD-SG (ports 22, 8080, 8081 from My IP)
GitHub repository	mohit-netflix-cicd
Docker Hub repository	mohit-netflix
Image name	YOUR-DOCKERHUB-USERNAME/mohit-netflix:latest
Jenkins job	CloudFlix-CICD-Pipeline
Jenkins credential ID	dockerhub-creds
Website URL	http://YOUR_EC2_PUBLIC_IP:8081

PHASE 1 · STEPS 1-3

Launch an AWS EC2 Instance

Step 1 Open the EC2 console

- Log in to the AWS Management Console.
- Search for EC2.
- Select Instances, then Launch instances.

Step 2 Configure the instance

Use this lab configuration:

Setting	Value
Name	Mohit-Jenkins-CICD
AMI	Amazon Linux 2023
Instance type	t3.medium (recommended for this combined lab)
Key pair	Create or select a key pair
Storage	20-30 GiB recommended
Network	A VPC and subnet with outbound internet access

> **NOTE · Sizing**
> Jenkins and Docker both run on this one machine, so allow enough memory and disk space for building images.

Step 3 Configure the security group

Create a security group named Jenkins-CICD-SG with these inbound rules:

Port	Type	Source	Purpose
22	SSH	My IP	Terminal access
8080	Custom TCP	My IP	Jenkins web UI
8081	Custom TCP	My IP	Website output

Security groups act as instance-level firewalls. Port 8080 is for Jenkins, and port 8081 serves your containerized website.

> **SECURITY · Do not expose Jenkins or SSH to the whole internet**
> Restrict access to your own public IP for this lab.

- Click Launch instance.
- Wait until the state is Running and the status checks pass.
- Copy the Public IPv4 address.

PHASE 2 · STEPS 4-9

Connect to EC2 and Install Jenkins

Step 4 Connect to Amazon Linux

Select the instance, click Connect, then use EC2 Instance Connect if available. Alternatively, connect over SSH from your terminal:

```
ssh -i your-key.pem ec2-user@YOUR_EC2_PUBLIC_IP
```

Replace the key filename and IP address with your own values.

Step 5 Update packages

```
sudo dnf update -y
```

Step 6 Install Java 21

Jenkins needs a compatible Java runtime. This lab uses Amazon Corretto 21.

```
sudo dnf install -y java-21-amazon-corretto
java -version
```

The output should show Java 21.

Step 7 Add the Jenkins repository

Run each command separately. First, download the repository file:

```
sudo wget -O /etc/yum.repos.d/jenkins.repo \
  https://pkg.jenkins.io/rpm-stable/jenkins.repo
```

Import the repository signing key:

```
sudo rpm --import \
  https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key
```

Install Jenkins:

```
sudo dnf install -y jenkins
```

Step 8 Start Jenkins

```
sudo systemctl enable jenkins
sudo systemctl start jenkins
sudo systemctl status jenkins
```

Confirm the service shows active (running) before continuing.

Step 9 Open the Jenkins UI

In your browser, open http://YOUR_EC2_PUBLIC_IP:8080. You should see the Jenkins unlock screen. Read the initial password from your EC2 terminal:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Paste the password into Jenkins, then:

- Click Continue, then Install suggested plugins.
- Create your admin account.
- Wait for installation to finish.
- Confirm the Jenkins URL and click Start using Jenkins.

> **CHECKPOINT · Checkpoint**
> You now have a working Jenkins dashboard.

PHASE 3 · STEPS 10-12

Install Git and Docker on EC2

Step 10 Install Git

```
sudo dnf install -y git
git --version
```

Jenkins uses Git to retrieve your website source code from GitHub.

Step 11 Install Docker Engine

On Amazon Linux 2023:

```
sudo dnf install -y docker
```

Start Docker and enable it at boot:

```
sudo systemctl enable --now docker
sudo systemctl status docker
```

Verify Docker:

```
sudo docker --version
sudo docker ps
```

These commands follow AWS's documented Docker installation approach for Amazon Linux 2023.

Step 12 Allow Jenkins to use Docker

Jenkins runs as its own Linux user, which needs permission to run Docker commands. For this single-instance lab, run:

```
sudo usermod -aG docker jenkins
sudo usermod -aG docker ec2-user
sudo systemctl restart jenkins
```

Reconnect your SSH session so the ec2-user group membership takes effect, then check Docker access:

```
docker ps
```

To confirm Jenkins itself can reach Docker, check the Jenkins service now and run the test build in a later step.

> **SECURITY · Docker group = root-level access**
> Membership in the docker group effectively grants root-level control of the host. This shortcut is for a disposable learning lab. In production, use an isolated build agent or a more restrictive container-build architecture.

PHASE 4 · STEPS 13-17

Create the GitHub Repository

Step 13 Create a new repository

- Open GitHub and click New repository.
- Repository name: **mohit-netflix-cicd**
- Choose Public for this beginner lab, so Jenkins can clone it without GitHub credentials.
- Create the repository.

Step 14 Prepare your project files

On your computer, create a folder containing two files:

```
mohit-netflix-cicd/
  index.html
  Dockerfile
```

Step 15 Create index.html

Paste the following into index.html. It is a responsive, Netflix-inspired streaming landing page built for this educational project.

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>CloudFlix | Streaming Demo</title>
 <style>
 * { box-sizing: border-box; }
 body {
 margin: 0;
 background: #101010;
```

```
color: white;
font-family: Arial, sans-serif;
}
header {
display: flex;
justify-content: space-between;
align-items: center;
padding: 22px 6%;
background: #080808;
}
.logo {
color: #e50914;
font-size: 30px;
font-weight: 900;
letter-spacing: 2px;
}
.sign-in {
background: #e50914;
padding: 10px 18px;
border-radius: 5px;
color: white;
text-decoration: none;
}
.hero {
min-height: 420px;
padding: 90px 7%;
background:
linear-gradient(90deg, #101010 10%, #101010d9 55%, #10101055),
radial-gradient(circle at 75% 30%, #7e1019, #241019 45%, #101010 80%);
}
.hero h1 {
max-width: 650px;
font-size: clamp(38px, 6vw, 64px);
margin: 0 0 18px;
}
.hero p {
max-width: 540px;
font-size: 19px;
line-height: 1.6;
color: #ddd;
}
.button {
display: inline-block;
margin-top: 18px;
padding: 14px 22px;
background: #e50914;
border-radius: 4px;
color: white;
text-decoration: none;
font-weight: bold;
}
main { padding: 28px 6%; }
.cards {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
gap: 16px;
}
```

```
 .card {
 min-height: 150px;
 padding: 22px;
 border-radius: 8px;
 background: linear-gradient(135deg, #4b1020, #202044);
 display: flex;
 align-items: end;
 font-weight: bold;
 font-size: 19px;
 }
 footer {
 padding: 28px;
 text-align: center;
 color: #aaa;
 }
 </style>
</head>
<body>
 <header>
 <div class="logo">CLOUDFLIX</div>
 <a class="sign-in" href="#shows">Explore</a>
 </header>
 <section class="hero">
 <h1>Stories worth staying up for.</h1>
 <p>
 Explore a world of movies, series, and unforgettable adventures.
 Built and deployed through a Jenkins CI/CD pipeline on AWS.
 </p>
 <a class="button" href="#shows">Explore the collection</a>
 </section>
 <main id="shows">
 <h2>Trending on CloudFlix</h2>
 <div class="cards">
 <div class="card">The Last Mission</div>
 <div class="card">Beyond Earth</div>
 <div class="card">Code Red</div>
 <div class="card">The Final Chapter</div>
 </div>
 </main>
 <footer>
 CloudFlix Demo · Built with Git, GitHub, Jenkins and Docker
 </footer>
</body>
</html>
```

Step 16 Create the Dockerfile

Create a file named exactly Dockerfile, with no extension:

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

Instruction	What it does
`FROM nginx:alpine`	Starts from a lightweight Nginx image
`COPY index.html ...`	Places your page in the Nginx web root

Instruction	What it does
`EXPOSE 80`	Documents the web port the container listens on

No separate build tool is needed to package this static website.

Step 17 Push the files to GitHub

Open Git Bash or a terminal in the project folder and run:

```
git init
git add index.html Dockerfile
git commit -m "Create CloudFlix landing page"
git branch -M main
git remote add origin https://github.com/YOUR-GITHUB-USERNAME/mohit-netflix-cicd.git
git push -u origin main
```

Replace YOUR-GITHUB-USERNAME with your GitHub username. If Git asks you to authenticate, follow GitHub's current authentication flow. Verify that both files appear in the repository.

PHASE 5 · STEPS 18-19

Prepare Docker Hub

Step 18 Create your Docker Hub repository

- Open Docker Hub and sign in, or create an account.
- Select My Hub, then Repositories, then Create repository.
- Repository name: **mohit-netflix**
- Set visibility to Public for this lab, then create the repository.

Your image name will be:

```
YOUR-DOCKERHUB-USERNAME/mohit-netflix:latest
```

Step 19 Create a Docker Hub access token

Use an access token rather than your Docker Hub account password.

- Open Docker Hub account settings and go to Personal access tokens.
- Create a token with the minimum permissions needed to push to your repository.
- Copy the token and store it securely.

> **SECURITY · Keep it secret**
> You will store this token in Jenkins credentials. Never put it in your GitHub repository, Dockerfile or shell script.

PHASE 6 · STEPS 20-21

Configure Jenkins Plugins and Credentials

Step 20 Install the required plugins

- From the dashboard, click Manage Jenkins, then Plugins, then Available plugins.
- Search for and install both of these:
  - **Git plugin** - lets Jenkins check out your repository.
  - **Credentials Binding plugin** - lets a build use stored secrets without hardcoding them.
- Restart Jenkins if prompted.

Step 21 Add Docker Hub credentials

- Go to Manage Jenkins, then Credentials.
- Open System, then Global credentials (unrestricted).
- Click Add Credentials and choose Username with password.
- Enter the values below, then click Create.

Field	Value
Username	Your Docker Hub username
Password	Your Docker Hub access token
ID	dockerhub-creds
Description	Docker Hub push credentials

Jenkins now stores the credentials so the job can use them securely.

PHASE 7 · STEPS 22-25

Create the Jenkins Freestyle Job

Step 22 Create a new Freestyle project

- From the dashboard, click New Item.
- Enter the job name **CloudFlix-CICD-Pipeline**
- Select Freestyle project and click OK.

Step 23 Configure General settings

Under General, add this description:

```
CI/CD pipeline to checkout CloudFlix source code from GitHub,
build and test a Docker image, push it to Docker Hub, and
deploy the container on AWS EC2.
```

Step 24 Connect Jenkins to GitHub

Under Source Code Management, select Git and fill in:

Field	Value
Repository URL	https://github.com/YOUR-GITHUB-USERNAME/mohit-netflix-cicd.git
Credentials	None (the repository is public)
Branch Specifier	*/main

Jenkins clones the source code into its workspace every time the job runs.

Step 25 Bind Docker Hub credentials

Scroll to Build Environment and:

- Enable Use secret text(s) or file(s).
- Add a binding of type Username and password (separated).
- Select the credential dockerhub-creds.
- Username variable: **DOCKERHUB_USER**
- Password variable: **DOCKERHUB_TOKEN**

> **HEADS UP · Option missing?**
> Verify that the Credentials Binding plugin is installed and that Jenkins has reloaded it.

PHASE 8 · STEPS 26-27

Add the CI/CD Build Script

Step 26 Add the build step

- Under Build Steps, click Add build step and select Execute shell.
- Paste the following script.

```
#!/bin/bash
set -eu

echo "==================================="
echo "STEP 1: Verify source code"
echo "==================================="
test -s index.html
test -s Dockerfile
echo "Website source files found."

echo "==================================="
echo "STEP 2: Build Docker image"
echo "==================================="
IMAGE="$DOCKERHUB_USER/mohit-netflix:latest"
docker build --pull -t "$IMAGE" .
echo "Docker image built successfully."

echo "==================================="
echo "STEP 3: Test the Docker image"
```

```
echo "====================================="
docker run -d --rm \
 --name cloudflix-test \
 -p 127.0.0.1:18081:80 \
 "$IMAGE"
trap 'docker stop cloudflix-test >/dev/null 2>&1 || true' EXIT
sleep 3
curl -fsS http://127.0.0.1:18081/ -o /tmp/cloudflix-test.html
grep -q "Stories worth staying up for" /tmp/cloudflix-test.html
echo "Website test passed."
docker stop cloudflix-test
trap - EXIT

echo "====================================="
echo "STEP 4: Login to Docker Hub"
echo "====================================="
printf '%s' "$DOCKERHUB_TOKEN" |
 docker login --username "$DOCKERHUB_USER" --password-stdin
echo "Docker Hub login successful."

echo "====================================="
echo "STEP 5: Push image to Docker Hub"
echo "====================================="
docker push "$IMAGE"
docker logout
echo "Image pushed successfully."

echo "====================================="
echo "STEP 6: Deploy container"
echo "====================================="
docker rm -f cloudflix || true
docker run -d \
 --name cloudflix \
 --restart unless-stopped \
 -p 8081:80 \
 "$IMAGE"

echo "====================================="
echo "CI/CD PIPELINE COMPLETED"
echo "====================================="
docker ps --filter name=cloudflix
```

HEADS UP · Before running

Ensure curl is installed (sudo dnf install -y curl), the Docker service is running, and the Jenkins service has Docker access.

The six stages of the script

#	Stage	What it does
1	Verify source	Checks that index.html and Dockerfile exist and are not empty
2	Build image	Builds the Docker image from the Dockerfile
3	Test image	Runs a temporary container and checks the page text
4	Docker Hub login	Authenticates using the bound credentials

#	Stage	What it does
5	Push image	Pushes the image to Docker Hub, then logs out
6	Deploy	Starts the website container on EC2, port 8081

Step 27 Save the job

Click Save. Your first end-to-end Freestyle CI/CD job is now configured.

PHASE 9 · STEPS 28-30

Run and Verify the Pipeline

Step 28 Execute the job

- Open CloudFlix-CICD-Pipeline and click Build Now.
- Wait for the build to finish.
- Under Build History, click the latest build number, then Console Output.

A successful run ends with lines like these:

```
Website source files found.
Docker image built successfully.
Website test passed.
Docker Hub login successful.
Image pushed successfully.
CI/CD PIPELINE COMPLETED
Finished: SUCCESS
```

The real output will also contain additional Docker and Jenkins logs.

Step 29 Verify the image in Docker Hub

Open Docker Hub, go to Repositories, open mohit-netflix and confirm that the latest tag exists.

Step 30 View the website

Open your browser at http://YOUR_EC2_PUBLIC_IP:8081. You should see your CloudFlix landing page.

> **HEADS UP · Browser can't connect?**
> Verify that TCP port 8081 is allowed in your EC2 security group from your IP address, and that the container is running. Use the commands below to investigate.

```
docker ps
docker logs cloudflix
curl -I http://localhost:8081
```

PHASE 10 · THE CHANGE-AND-REBUILD LOOP

Test the CI/CD Loop

Your first run was manual. To see continuous integration in action, make a small change to index.html on your computer.

Step A Change, commit and push

Edit a line of text, then run:

```
git add index.html
git commit -m "Update CloudFlix landing page"
git push origin main
```

Step B Rebuild in Jenkins

- In Jenkins, open the job and click Build Now again.
- Confirm Jenkins checks out the updated code, rebuilds the image, pushes it to Docker Hub, and redeploys the container.
- Refresh your browser on port 8081 to see the change.

> **NOTE · Want builds on every push?**
> Configure a GitHub webhook and the matching Jenkins trigger. Manual builds are enough for this first lab.

WRAP-UP

Final Architecture and Checklist

1	2	3	4	5	6
Developer	**GitHub**	**Jenkins Job**	**Docker Hub**	**EC2 + Docker**	**Browser**
Edit, commit, push	Source repository	Checkout, build, test, push, deploy	Stores the image	Runs the container	CloudFlix on 8081

Completion checklist

Done?	Check
[ ]	EC2 instance is running and reachable over SSH from My IP
[ ]	Jenkins dashboard opens on port 8080
[ ]	Git and Docker installed; jenkins user can run docker
[ ]	index.html and Dockerfile are visible in the GitHub repository
[ ]	Docker Hub repository mohit-netflix exists with an access token
[ ]	dockerhub-creds saved in Jenkins and bound in the job
[ ]	Build Now ends with Finished: SUCCESS
[ ]	The latest tag is visible on Docker Hub

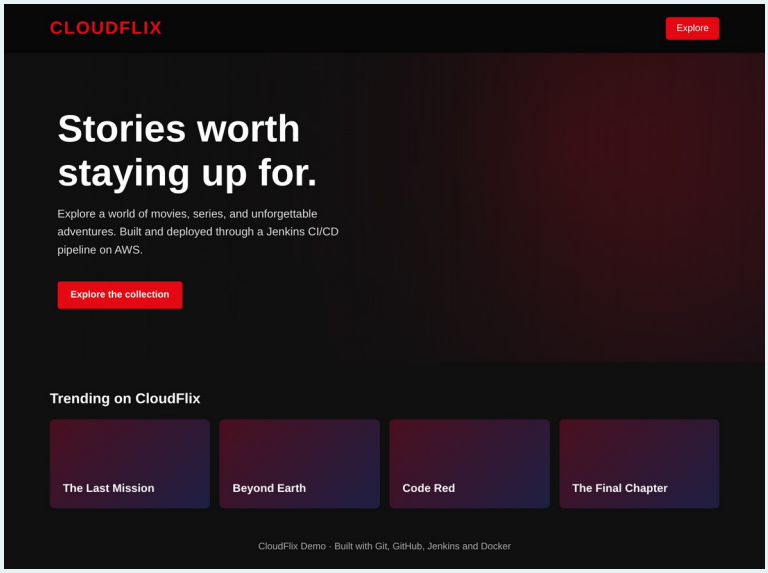
Done?	Check
[ ]	CloudFlix loads at http://YOUR_EC2_PUBLIC_IP:8081
[ ]	A pushed change appears after a second build

Common problems

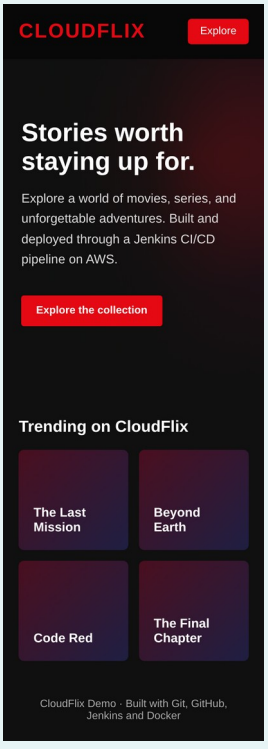
Symptom	Likely cause and fix
Jenkins page does not load	Port 8080 not allowed from your IP, or the service is not running. Check the security group and sudo systemctl status jenkins.
permission denied on docker	Jenkins was not restarted after the usermod command. Re-run Step 12 and restart Jenkins.
Binding option missing in the job	Credentials Binding plugin not installed or Jenkins not reloaded (Step 20).
Push is denied	Wrong token, wrong username, or the repository name does not match mohit-netflix.
Website test fails in the build	curl missing, or the container did not start in 3 seconds. Check docker logs and install curl.
Site not reachable on 8081	Port 8081 not open to your IP, or the cloudflix container is not running (docker ps).

Expected Output: CloudFlix Landing Page

After a successful build, opening http://YOUR_EC2_PUBLIC_IP:8081 shows the page below. The left image is the desktop view and the right image is the mobile view of the same responsive page.



Desktop view (1280 px wide)



Mobile view (390 px wide)

HEADS UP · Important production note

This single-instance design is for learning. Jenkins and the application share one EC2 host, so it is not a production-grade architecture. For production, use a dedicated Jenkins controller with isolated build agents, secure Jenkins with HTTPS and restricted access, and deploy the container to a separate target such as ECS or EKS.

Official references

- Jenkins on AWS
- Docker installation on Linux
- Jenkins credentials

Created by **Mohit Gadilohar**
www.linkedin.com/in/mohit-gadilohar-devops